COMPUTER SYSTEMS AND PLATFORM TECHNOLOGIES

IT0047

FEU ALABANG FEU DILIMAN FEU TECH

Technology Driven by Innovation

MODULE 5

8086/8088 Instruction Sets

MODULE 5.3 Part 1

Flag Control and Control Transfer Instructions

At the end of the lesson, the student should be able to :

- **Identify the Flag Control and Control Transfer Instruction set**
- **Identify the value of the destination operand and conditional flags after execution of Flag Control and Control Transfer Instructions**

FLAG CONTROL INSTRUCTIONS

- The 8086/8088 provides a group of instructions that directly affects the state of the flags.

- The LAHF Instruction

- The LAHF (***Load AH from flags***) instruction copies the SF, ZF, AF, PF, and CF into bits b7, b6, b4, b2, and b0 respectively of register AH.

Format: LAHF

Action: AH ⟵ SF ZF x AF x PF x CF

Pointers:

- Bits b5, b3, and b1 of register AH are undefined after execution.
- Example: Assume the value of the following flags: SF = 1, ZF = 0, AF = 1, PF = 0, CF = 1
- After executing the LAHF instruction, AH will have the value of 10x1x0x1.
- Assuming the undefined bits have a value of 0, then AH is equal to 91H.

The SAHF Instruction

- The SAHF (***Store AH into flags***) instruction transfers bits b7, b6, b4, b2, and b0 from register AH into SF, ZF, AF, PF, and CF, respectively.

Format: SAHF

Action: SF ZF x AF x PF x CF $\longrightarrow$ AH

Example:

MOV AH, 25H

SAHF

- After executing the SAHF instruction, the value of the flags will be:

0	0	1	0	0	1	0	1
SF	ZF		AF		PF		CF

The CLC Instruction

- The CLC (***Clear Carry Flag***) instruction clears the carry flag.

Format: CLC

Action: CF $\longleftarrow$ 0

The STC Instruction

- The SLC (***Set Carry Flag***) instruction sets the carry flag to 1.

Format: STC

Action: CF $\longleftarrow$ 1

The CMC Instruction

- The CMC (***Complement Carry Flag***) instruction complements the carry flag.

Format: CMC

Action: $CF \leftarrow \overline{[CF]}$

The PUSHF Instruction

- The PUSHF (***Push Status Register onto Stack***) instruction pushes the status register onto the stack.
- PUSHF decrements SP by 2 and then moves the status register to the top of the stack.

Format: PUSHF

Action: SP ← SP - 2

SP ← [status register]

The POPF Instruction

- The POPF (***Pop Flags off the Stack***) instruction transfers the word pointed to by SP to the status register, thereby altering the value of the flags.

Format: POPF

Action:

status register ← [SP]

SP ← SP + 2

MODULE 5.3 Part2

Basic 8086/8088 Instruction Sets: Control Transfer Instructions

At the end of the lesson, the student should be able to :

- **Identify the Control Transfer Instruction sets**
- **Identify the values of the conditional flags**

CONTROL TRANSFER INSTRUCTIONS

- The code segment and IP keep track of the next instruction to be executed. This means that programs are executed sequentially.
- However, there are times when there is a need to alter the flow of program execution such as program loops or function calls.
- The instructions that can alter the flow of a program are ***iteration control instructions***, ***conditional transfers***, ***unconditional transfers***, and ***subroutines***.

The JMP Instruction

- The JMP (***Unconditional Jump***) instruction unconditionally transfers control to the target location.

Format: JMP operand

Action: Jump is initiated to the address specified by the operand.

Valid Format

Operand	Example
Short-label	JMP short L1
Near-label	JMP L1
Far-label	JMP far L1
Regptr16	JMP DX
Memptr16	JMP [BX]
Memptr32	JMP far [BX]

Pointers:

1. The JMP Instruction can either be ***Intrasegment Jump*** or ***Intersegment Jump***. In intrasegment jump, the jump is limited within the code segment only. Only the value of IP is modified. For intersegment jump, the jump is allowed to any other code segment. This means both CS and IP are modified.

2. Operands may be labels (representing the address of the target location).

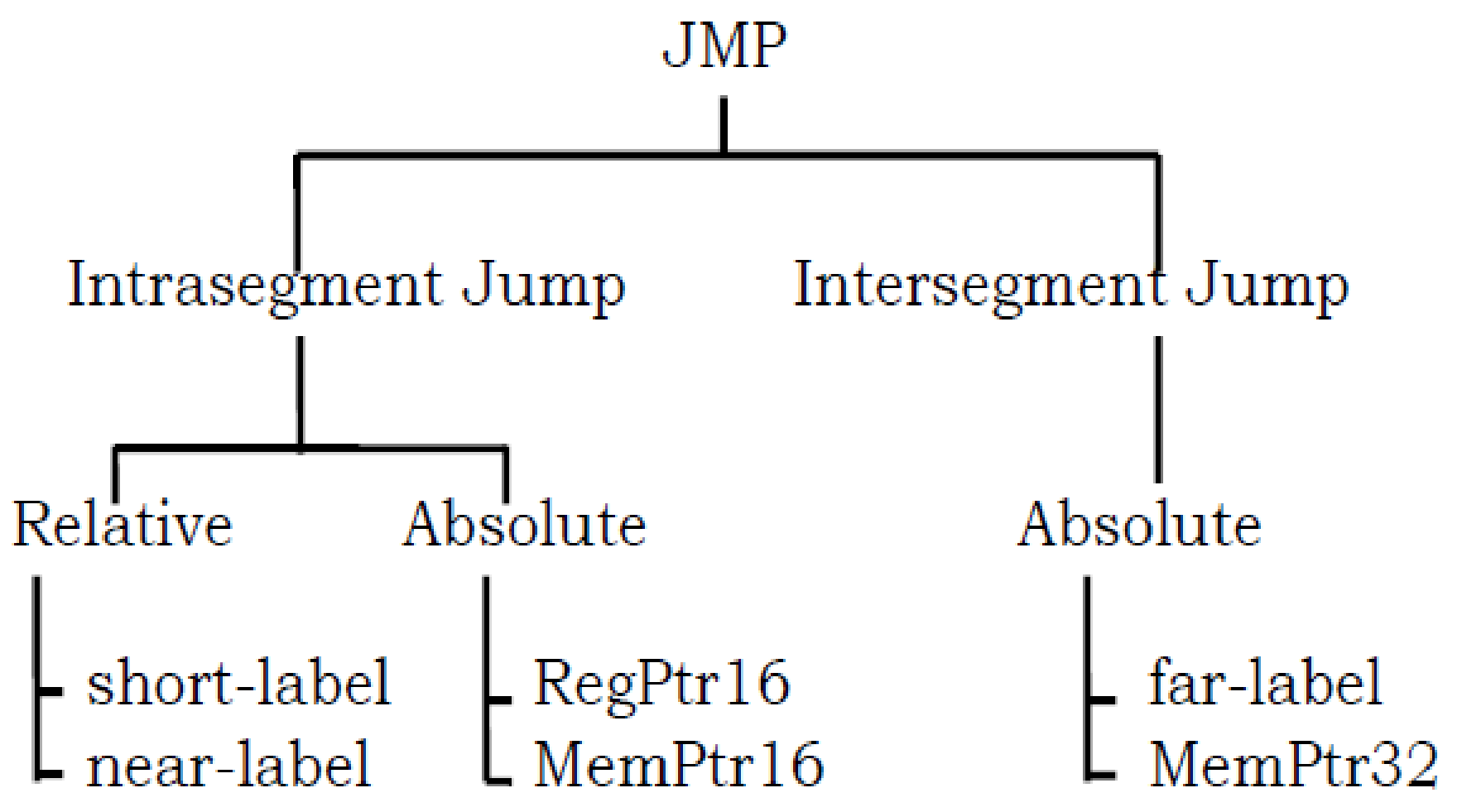
3. Operands could either be ***relative*** or ***absolute***. In relative operand, the operand is a signed displacement (or offset) from the JMP instruction to the target location. This means that the address of the target location is obtained by adding the displacement to IP. In the absolute operand, the operand itself is the address of the target location.

continuation

4. Intrasegment jumps support both absolute and relative operands, while intersegment jumps support absolute operands only.

5. Relative operands for intrasegment jump could either be ***short-label*** or ***near-label***. For short-label, program transfers control within -128 to 127 bytes from the location of the jump instruction (the displacement operand is only 1 byte). The prefix *short* signifies a short-label jump. For near-label, program transfers control within -32,768 to 32,767 bytes from the location of the jump instruction (the displacement operand has 2 bytes).

Summary of JMP



Examples:

1. Short-label

```
    MOV AX, 0001H
    JMP short L1
    MOV BX, 0002H
L1: MOV BX, 0001H
```

- After execution, BX will contain 0001H. In terms of execution, both short-label jump and near-label jump are the same. The only difference is the distance of the jump. Near label jump could jump farther than short-label jump. Note that the jump could either be a forward jump or a backward jump.

Example

2. Far-Label

```
MOV AX, 0001H
JMP far L1
MOV BX, 0002H
MOV BX, 0001H
.
L1: MOV BX, 0003H
```

- Label L1 is assumed to be on another code segment.

The prefix far signifies an intersegment jump.

Example

3. RegPtr16

```
MOV BX, 0000H
JMP BX
```

- In this example, the JMP instruction will jump to absolute address CS:0000.
- This address is pointed by register BX. Since this is an intrasegment jump, BX contains the new IP value.

Example

4. MemPtr16

MOV BX, 0000H

JMP [BX]

DS:0001H	01H
DS:0000H	00H

- In this example, the JMP instruction will jump to absolute address CS:0100.
- This address is located in the memory pointed by register BX.
- Since this is an intrasegment jump, the memory contains the new IP value.

Example

5. MemPtr32

MOV BX, 0000H

JMP far [BX]

DS:0003H	01H
DS:0002H	00H
DS:0001H	02H
DS:0000H	00H

- In this example, the JMP instruction will jump to absolute address 0200:0100. The address is located in the memory pointed by register BX. Since the JMP instruction is an intersegment jump, that memory contains both the new CS and IP values.

The Jcc Instructions

- The J*cc* (***Conditional Jump***) instructions are jumps that may or may not transfer control depending on the state of the status flags at the time the instruction is executed.

Format: J*cc* operand

Action: If the specified condition *cc* is true, it will jump to the address specified in the operand; otherwise the next instruction is executed.

Comparing Unsigned Numbers

Mnemonic	"Jump if ..."	Condition
JA/JNBE	above/not below nor equal	(CF or ZF) = 0
JAE/JNB	above or equal/not below	CF = 0
JB/JNA	below/not above nor equal	CF = 1
JBE/JNA	below or equal/not above	(CF or ZF) = 1

Comparing Signed Numbers

Mnemonic	"Jump if ..."	Condition
JG/JNLE	greater than/not less than or equal	ZF=0 or (SF=OF)
JGE/JNL	greater than or equal/not less than	SF = OF
JL/JNGE	less than/not greater than or equal	SF ≠ OF
JLE/JNG	less than or equal/not greater than	ZF=1 or (SF≠OF)

Test Flags

Mnemonic	"Jump if ..."	Condition
JE/JZ	equal to/zero	ZF = 1
JNE/JNZ	not equal to/not zero	ZF = 0
JC	carry	CF = 1
JNC	not carry	CF = 0
JS	sign	SF = 1
JNS	not sign	SF = 0
JP/JPE	parity/parity even	PF = 1
JNP/JPO	no parity/parity odd	PF = 0
JO	overflow	OF = 1
JNO	no overflow	OF = 0

Pointers:

1. Only the short-label operand is supported by the conditional jump instructions. This means that the program could only transfer program control within -128 to +127 bytes from the location of the conditional jump instruction.
2. Conditional jump should be placed immediately after an instruction that will modify the status flags (i.e., CMP, INC, DEC, etc.).
3. To circumvent the limitation of the short label, an unconditional jump is used around the conditional jump.

Examples:

1. MOV AL, A5H
MOV BL, 73H
CMP AL, BL
JA L1

The JA L1 instruction is read as “if AL is higher than BL, then jump to L1.” The program will jump to *L1* since the value of AL (A5H or 165 in decimal) is higher than the value of BL (73H or 115 in decimal). Remember that the values are treated as unsigned numbers.

Example

2. MOV AL, A5H
   MOV BL, 73H
   CMP AL, BL
   JG L1

The JG L1 instruction is read as "if AL is greater than BL, then jump to L1." The program will not jump to *L1* since the value of AL (A5H or -91 in decimal) is not greater than the value of BL (73H or +115 in decimal). Remember that the values are treated as signed numbers.

Example

2. MOV AL, A5H
   MOV BL, 73H
   CMP AL, BL
   JG L1

The JG L1 instruction is read as "if AL is greater than BL, then jump to L1." The program will not jump to *L1* since the value of AL (A5H or -91 in decimal) is not greater than the value of BL (73H or +115 in decimal). Remember that the values are treated as signed numbers.

Explanation

- When the program is executed, the value of AL will become equal to 02H. Then the value of CL will be decremented by 1. Its value will therefore be equal to 02H. The JNZ AGAIN instruction will check the value of CL if it is zero or not (by checking the zero flag).
- If it is not equal to 0, the program will jump back to AGAIN. Since CL is not zero, the program will go back to the ADD AL, 02H instruction again. The value of AL will now become equal to 04H. Then the value of CL will again be decremented by 1.
- Its value will therefore be equal to 01H. Since it is still not equal to zero, the program will go back to the ADD AL, 02H instruction again. The value of AL will now become equal to 06H. Then the value of CL will again be decremented by 1. Its value will therefore be equal to 00H. Since it is now equal to zero, the program will not jump. Instead it will proceed to the next instruction after the JNZ AGAIN instruction.

Example

4. MOV AL, 7FH
   MOV BL, 75H
   ADD AL, BL
   JO NEXT

The JO NEXT instruction will cause the program to jump to NEXT if there was an overflow condition. After executing the ADD AL, BL instruction, the overflow flag will become equal to 1 indicating that there was an overflow. Therefore, the program will jump to NEXT.

Example

5. MOV AL, 01H
   DEC AL
   JG L1

- When a conditional jump is placed after an instruction having one operand, the comparison is between the operand and 0.

- In this example, the *JG L1* instruction is read as "decrement the value of AL by one, if AL is greater than 0, then jump to *L1*." After decrementing AL, it now has a value of 0. Therefore, the program will not jump to *L1* since 0 is not greater than 0.

Example

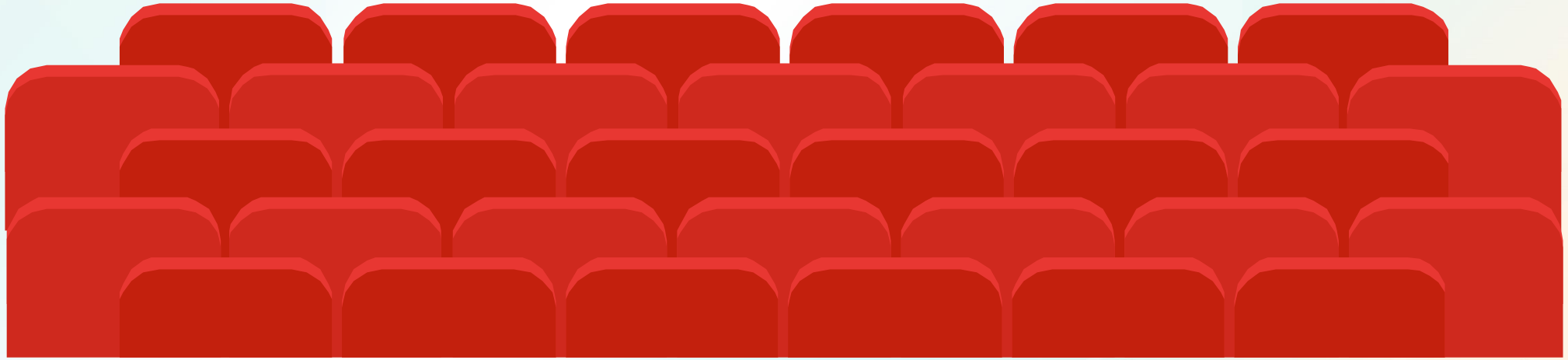
6. Circumventing the limitation of the short-label:

```
MOV BX, 0002H
MOV CX, 0002H
CMP BX, CX
JE L1
      :
      :
      :
L1:   :
      :
```

Explanation

If there are 1,000 lines in between the JE instruction and L1, the assembler will issue an error message since the operand is only a short-label.

Q&A SESSION

ASK ANY QUESTION RELATED TO OUR TOPIC FOR TODAY.

THANK YOU